Лабораторная работа № 4

Тема: Отладка Express, логирование запросов и requestId

Цель: Научиться просматривать запросы в терминале, сопоставлять requestId с X-Request-Id и использовать точку останова Chrome DevTools.

Дисциплина: Node.js и Express.js. РО 4.5

Лабораторная работа № 4. Отладка и логирование Node.js-приложений

Тема: просмотр HTTP-запросов в терминале, requestId и точка останова в Chrome DevTools.

Цель: научиться видеть запросы Express в логах, находить один и тот же requestId в терминале и HTTP-ответе, а также ставить точку останова без изменения кода.

Перед началом

Продолжаем существующий проект C:\WORK\js\leson8prisma. Express и PostgreSQL должны работать. Не создаём новую БД, не меняем данные и не переписываем существующие маршруты.

> **Обратите внимание**
>
> В этой работе сервер запускается командой node server.js в терминале VS Code. Если в проекте запускающим файлом служит app.js, адаптируйте команду к реальной структуре. На вашем проекте server.js уже успешно выводил Server started — используйте именно его.

1. Проверьте установку Pino

Откройте терминал VS Code (Терминал → Создать терминал). В папке проекта выполните команды, если Pino и pino-pretty ещё не установлены:

```
npm install pino
npm install --save-dev pino-pretty
```

2. Проверьте файл config/logger.js

Откройте config/logger.js. Если он уже создан и работает, не переписывайте его. Для сравнения приведён простой рабочий вариант:

```
import pino from "pino";

export const logger = pino({
    level: "info",
    redact: ["password", "accessToken", "req.headers.authorization"],
    transport: {
        target: "pino-pretty",
        options: { colorize: true }
    }
});
```

Pino записывает сообщения. pino-pretty делает их удобными для чтения в терминале. В этом учебном варианте transport используется для локальной разработки.

3. Создайте middleware/requestContext.js

Создайте файл middleware/requestContext.js. Он назначает отдельный номер (requestId) каждому HTTP-запросу и передаёт этот номер в ответ браузеру.

```
import { randomUUID } from "node:crypto";
import { logger } from "../config/logger.js";

export function requestContext(req, res, next) {
    const requestId = randomUUID();
    req.requestId = requestId;
    req.log = logger.child({ requestId });
    res.set("X-Request-Id", requestId);
    next();
}
```

randomUUID() генерирует идентификатор. logger.child({ requestId }) автоматически добавляет его к сообщениям данного запроса. Код демонстрационный: здесь клиент не задаёт ID самостоятельно.

4. Создайте middleware/requestLogger.js

Создайте файл middleware/requestLogger.js. Он выводит в терминал начало и завершение каждого запроса.

```
export function requestLogger(req, res, next) {
    req.log.info({
        method: req.method,
        path: req.originalUrl
    }, "Request started");

    res.on("finish", () => {
        req.log.info({
            method: req.method,
            path: req.originalUrl,
            statusCode: res.statusCode
        }, "Request completed");
    });

    next();
}
```

Первое сообщение выводится при получении запроса. Второе — после отправки ответа клиенту. Оно содержит HTTP-статус.

5. Подключите оба middleware в app.js

Откройте корневой файл app.js. В начале файла, среди других import, ДОБАВЬТЕ следующие два импорта:

```
import { requestContext } from "./middleware/requestContext.js";
import { requestLogger } from "./middleware/requestLogger.js";
```

Теперь найдите строку app.use(express.json()); (или аналогичную настройку JSON). СРАЗУ ПОСЛЕ неё, но обязательно ПЕРЕД app.use("/api/products", ...) и другими маршрутами, добавьте:

```
app.use(requestContext);
app.use(requestLogger);
```

Обратите внимание

Не создавайте второй const app = express(), не удаляйте ваши импорты, маршруты, CORS или обработчики ошибок. Порядок важен: requestContext → requestLogger → маршруты. Если requestLogger подключён ПОСЛЕ маршрутов, успешные запросы могут до него не доходить.

6. Остановите старый сервер и запустите новый

Перейдите в терминал VS Code, в котором запущен сервер. Нажмите Ctrl + C. Затем выполните:

```
cd C:\WORK\js\leson8prisma
node server.js
```

Должны появиться сообщения вроде Application starting и Server started. Терминал оставьте открытым: именно в нём будут появляться server logs.

Обратите внимание

Просто запустить node server.js недостаточно: если requestContext и requestLogger не подключены в app.js, запросы не появятся в терминале. Если сообщение Server started есть, но HTTP-запросы отсутствуют, сначала проверяйте шаг 5. Также убедитесь, что запрос обращается к тому же серверу на localhost:3000.

7. Отправьте GET-запрос к реальной базе данных

В другом окне VS Code (например, в установленном HTTP-клиенте) или в обычной вкладке браузера выполните:

```
GET http://localhost:3000/api/products/1
```

Это GET-запрос, поэтому тело Body указывать не нужно. При наличии товара ID=1 в ответе придёт JSON товара из PostgreSQL. Важно: для проверки логирования подходит даже ответ 404, если маршрут существует.

8. Найдите requestId в терминале VS Code

Вернитесь к терминалу, в котором работает node server.js. Ниже пример, а не фактический вывод вашего приложения:

```
INFO: Request started
    requestId: "11111111-2222-4333-8444-555555555555"
    method: "GET"
    path: "/api/products/1"
```

```
INFO: Request completed
    requestId: "11111111-2222-4333-8444-555555555555"
    method: "GET"
    path: "/api/products/1"
    statusCode: 200
```

Один и тот же requestId в обеих записях означает, что они относятся к одному запросу. Конкретный идентификатор у вас будет другим.

9. Найдите X-Request-Id в HTTP-ответе

В HTTP-клиенте VS Code откройте именно RESPONSE → Headers (заголовки ответа), а не вкладку Headers в редакторе отправляемого запроса. Найдите X-Request-Id. Он должен совпадать с requestId в терминале.

В качестве альтернативы откройте ВТОРОЙ терминал VS Code, оставив первый с сервером открытым, и выполните:

```
curl.exe -i http://localhost:3000/api/products/1
```

В выводе HTTP-ответа найдите строку X-Request-Id: ... . Сравнивайте значение с последней парой логов в ПЕРВОМ терминале. Команда curl.exe создаёт новый запрос, поэтому его ID будет отличаться от предыдущего запроса браузера.

10. Проверьте, что каждому запросу назначается свой ID

Повторите команду curl.exe -i второй раз. В первом терминале появятся ещё две записи, а во втором — новый заголовок X-Request-Id. Для двух разных запросов ID должны различаться.

11. Поставьте точку останова без изменения исходного кода

Остановите сервер клавишами Ctrl + C. Перезапустите его в режиме отладки:

```
node --inspect server.js
```

В Google Chrome откройте chrome://inspect, найдите процесс server.js, нажмите inspect. В окне DevTools → Sources нажмите Ctrl + P и найдите файл с вашей функцией получения товара (например, productService.js или файл repository). Откройте его и щёлкните по номеру строки с вызовом Prisma findUnique()/findFirst() — это точка останова.

Теперь из обычного браузера либо HTTP-клиента отправьте GET http://localhost:3000/api/products/1. При достижении строки выполнения Node.js остановится. В DevTools справа откройте Scope → Local и посмотрите ID товара. Нажмите F10 в DevTools, чтобы выполнить строку, и F8, чтобы продолжить выполнение. Если ответ браузера ожидает завершения, это нормально — сервер пока остановлен на breakpoint.

> **Обратите внимание**
>
> F8/F10 нажимайте в окне Chrome DevTools, НЕ в VS Code. Если ваша функция реализована через другой файл/метод Prisma, установите breakpoint в реальной строке обращения к БД; предложенные имена файлов — примеры.

12. Самостоятельное задание

Выполните один GET-запрос http://localhost:3000/api/products/1 и самостоятельно найдите его requestId в двух местах: (1) в терминале сервера и (2) в заголовке X-Request-Id ответа. Запишите оба значения и убедитесь, что они совпадают. Затем повторите запрос и покажите, что новый ID отличается от предыдущего.

Результат лабораторной

В терминале есть сообщения Request started и Request completed с одинаковым requestId для одного запроса. В HTTP-ответе присутствует X-Request-Id с тем же значением. Студент умеет запускать сервер командой node server.js и ставить breakpoint в DevTools без изменения исходного кода.